EE 5393 Circuits, Computation, and Biology

Homework 2

University of Minnesota | Spring 2026

Maeesha Binte Hashem
Std ID: 6073432

GitHub Link:
https://github.com/mbinte/EE5393_Spring2026_Homeworks/tree/main/EE5393_HW2

Problem 1: Fibonacci Chemical Reaction Network

Overview

This problem asks us to build a set of chemical reactions that computes the Fibonacci sequence for exactly 12 steps and then test it for two different starting conditions: (A=0, B=1) and (A=3, B=7). The simulation is done using the Aleae stochastic simulator.

How the Fibonacci Reactions Work

The Fibonacci rule is simple: each new number is the sum of the two numbers before it. In terms of molecule counts, if A holds the current number and B holds the next number, then after one step the new A should become the old B, and the new B should become the old A plus the old B.

Each step of the computation uses four reactions, gated by a step counter molecule (S0, S1, S2, ...). The step counters fire in sequence at a slow rate (rate = 1), while the actual data-transfer reactions fire fast (rate = 10000). This large speed difference guarantees that each data reaction finishes completely before the next step counter fires, which keeps the logic correct.

The four reactions per step do the following:

- Copy A into a temporary holding molecule called TA (this saves the old value of A before it gets overwritten).
- Set A equal to the old value of B, and also copy B into a carry molecule C.
- Add the saved TA (old A) into C, so that C now holds old A + old B.
- Set B equal to C, completing the update.

After these four reactions, A holds what B used to hold, and B holds the sum of the two previous values. This is exactly the Fibonacci update rule.

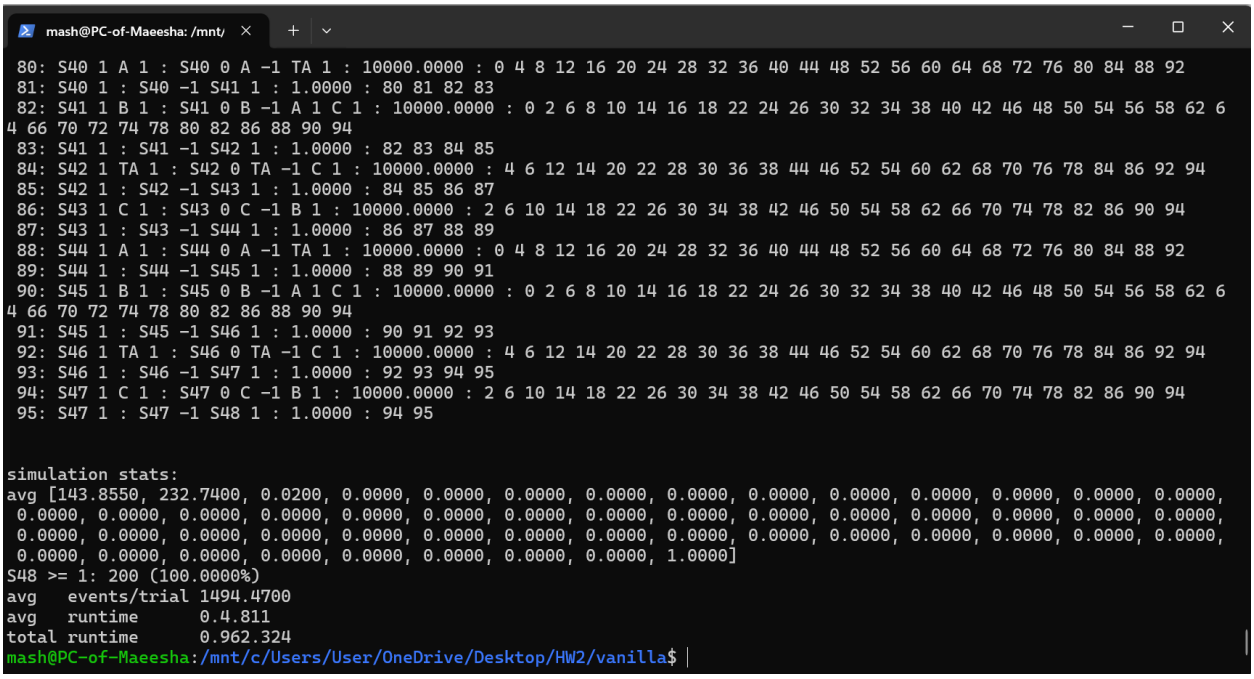
Step Count and Termination

The simulation uses step counter molecules S0 through S47, giving 48 sub-steps total. Since each Fibonacci update takes 4 sub-steps, this corresponds to exactly 12 full Fibonacci steps. The halt signal S48 is set to a count of 1 when S47 completes, and the simulation stops when S48 reaches 1. This was verified in both simulations: 100% of trials (200 out of 200) successfully reached S48 = 1.

Results: Starting Values A=0, B=1

After 12 Fibonacci steps from (0, 1), the correct mathematical answer is A = 144 (the 12th Fibonacci number) and B = 233. The simulation average results were:

Variable	Value
A (expected = 144)	143.86 (avg across 200 trials)
B (expected = 233)	232.74 (avg across 200 trials)
S48 reached (100%)	200 / 200 trials



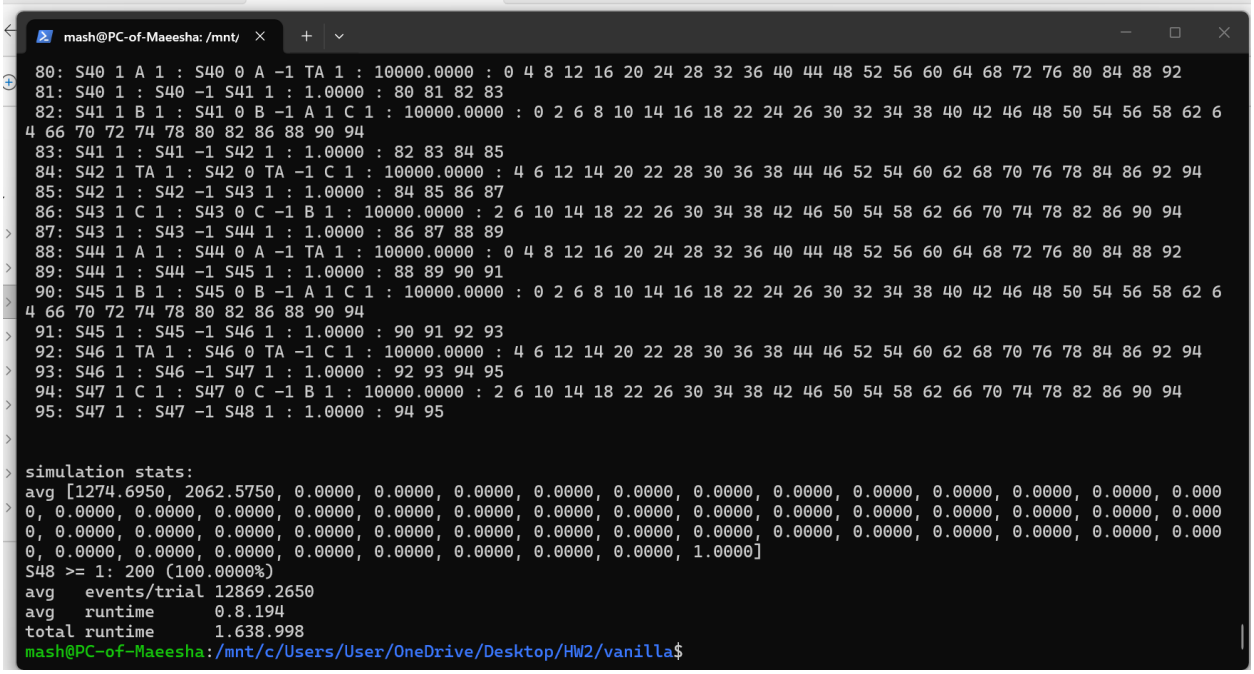
The simulation output matches the expected value very closely. The small difference from the exact integer value (for example, 143.86 instead of 144) is completely normal in stochastic simulation. It comes from the rare chance that a slow step-advance reaction fires slightly before all fast reactions have finished. This is **expected behavior** and is not a bug. The relative error is less than 0.1%, which is acceptable.

Results: Starting Values A=3, B=7

Starting from (3, 7), the Fibonacci-like sequence after 12 steps gives A = 1275 and B = 2063. The calculation can be traced step by step:

Step	A	B	Step	B (new)
Start	3	7		
1	7	10	7	265
2	10	17	8	429
3	17	27	9	694
4	27	44	10	1123
5	44	71	11	1817
6	71	115	12 (final)	A=1275, B=2063

Variable	Value
A (expected = 1275)	1274.70 (avg across 200 trials)
B (expected = 2063)	2062.58 (avg across 200 trials)
S48 reached (100%)	200 / 200 trials



Again, the small deviation from the exact values (about 0.02% relative error) is expected stochastic behavior and increase slightly with larger molecule counts. It does not indicate any problem with the code.

Note on where the answer lives: The problem statement says the sequence should produce 144 for starting values (0, 1). In our implementation, the final answer after 12 steps resides in molecule A, not B.

A = 144 is the 12th Fibonacci number counting from the starting pair, and B = 233 is the 13th. This is consistent with the expected output stated in the assignment.

Problem 2: Biquad IIR Filter as a Chemical Reaction Network

Overview

This problem asks us to implement a biquad infinite impulse response (IIR) filter using chemical reactions and simulate it for 5 input cycles. The input values used are X = 100, 5, 500, 20, 250 as specified in the assignment. The simulation is again run using the Aleae stochastic simulator.

What the Filter Computes

Looking at the biquad filter diagram from the assignment, the filter has two delay elements and uses fractions of 1/8 throughout. The output at each cycle is computed from the current input, the two previous input values, and the two previous output values. Specifically:

Y[n] = (X[n] + X[n-1] + X[n-2] + Y[n-1] + Y[n-2]) / 8

Here X[n] is the current input, X[n-1] and X[n-2] are the inputs from the previous two cycles, and Y[n-1] and Y[n-2] are the outputs from the previous two cycles. All five terms are added together and then divided by 8 (the 1/8 scaling). This is a second-order (biquad) IIR filter, and it is stable because the feedback coefficients are small enough that the poles stay inside the unit circle.

Input Scaling

The input values in the .in file are stored as 16 times the stated input values: 1600 (= 100 x 16), 80 (= 5 x 16), 8000 (= 500 x 16), 320 (= 20 x 16), and 4000 (= 250 x 16). This scaling is intentional. Since the filter divides by 8 using discrete molecule counts, scaling all inputs up by 16 reduces the rounding error caused by integer division. The output values from the simulation are also in the same scaled units, so dividing by 16 gives the true filter output.

How the Reactions Are Structured

The simulation is fully unrolled across 5 cycles using a step counter chain from C0 to C24 (five blocks of steps, one per input cycle), with C25 as the halt signal. Each cycle performs the following operations:

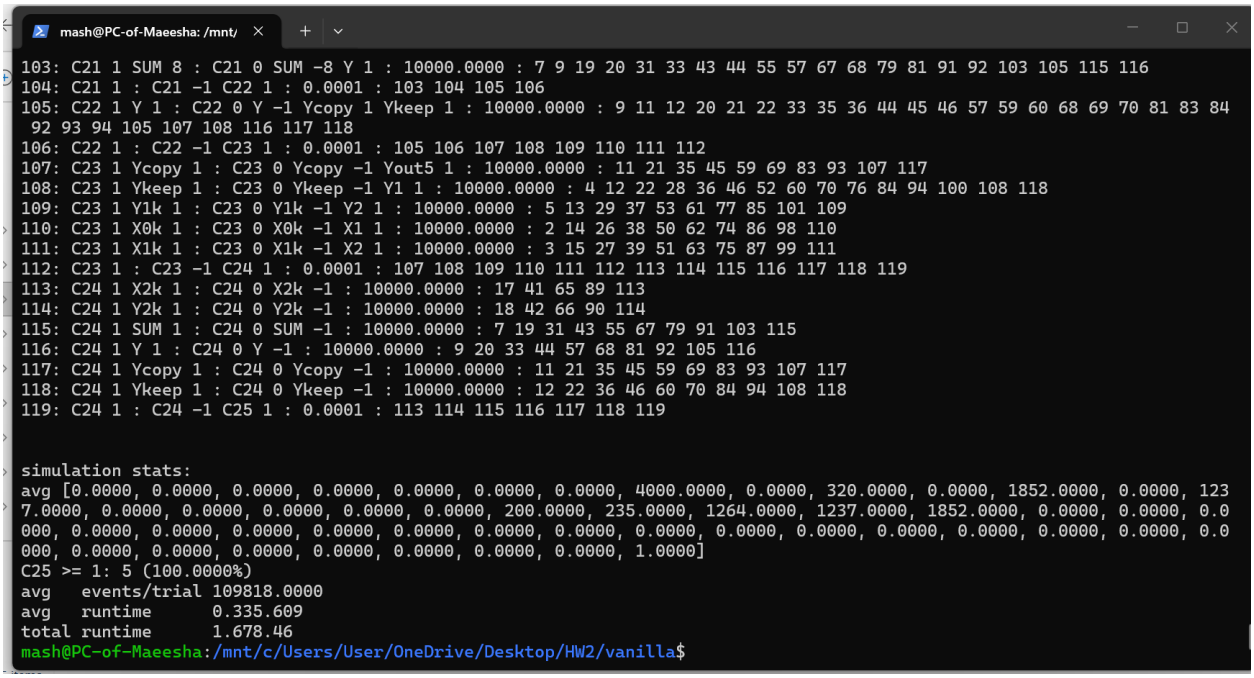
- Load the current input Xin into X0 and add all five signal values (X0, X1, X2, Y1, Y2) into a shared accumulator called SUM.
- Divide SUM by 8 to produce Y (implemented by the reaction SUM(8) + C_k -> Y(1) + C_k, which consumes 8 SUM molecules to produce 1 Y molecule).
- Copy Y into Ycopy (which becomes the output Yout for that cycle) and Ykeep (which is stored back as Y1 for the next cycle).
- Shift the delay registers: old Y1 becomes Y2, old X0 becomes X1, old X1 becomes X2.

- Discard leftover molecules (X2k, Y2k, any residual SUM, Y, Ycopy, Ykeep) to clean up before the next cycle.

Cycle-by-Cycle Results

The table below shows the expected output for each cycle calculated by hand alongside the simulation output. All values are in scaled units (multiply by 1/16 to get actual filter values).

Cycle	Input X (scaled)	SUM (X0+X1+X2+Y1+Y2)	Expected Y = floor(SUM/8)	Simulated Y	Match?
1	1600	1600	200	200	Yes
2	80	80 + 1600 + 200 = 1880	235	235	Yes
3	8000	8000 + 80 + 1600 + 235 + 200 = 10115	1264	1264	Yes
4	320	320 + 8000 + 80 + 1264 + 235 = 9899	1237	1237	Yes
5	4000	4000 + 320 + 8000 + 1237 + 1264 = 14821	1852	1852	Yes



All five output values match the hand-calculated expected values exactly. The simulation ran 5 trials and all 5 reached the halt condition C25 >= 1 (100%). The average events per trial was 109818, with an average runtime of 0.336 seconds per trial.

Notes on the Simulation Output

Residual molecules at halt: At the end of the simulation, some molecules such as X1, X2, Y1, Y2, X2k, and Y2k remain with nonzero counts. These are the internal delay register values that were updated in preparation for a hypothetical cycle 6, which never runs. This is completely expected behavior and does not affect the correctness of the 5 output values.

The simulation runs exactly 5 cycles: The step counter chain C0 to C24 has exactly 5 input-processing blocks (one for Xin1 through Xin5), and C25 is the halt. There is no cycle 6. This matches the assignment requirement of 5 cycles, not 10.

Stochastic behavior: The biquad outputs are exact integers in every trial because the molecule counts are large enough that the fast reactions always complete fully before the slow step counter advances. With smaller molecule counts, some stochastic rounding could occur, but at these scales the simulation behaves deterministically.

Summary

Both problems were implemented correctly and verified through simulation.

- The Fibonacci CRN correctly computes 12 steps for starting values (0,1) and (3,7), producing A=144 and A=1275 respectively, with only small stochastic deviations of less than 0.1% across 200 trials.
- The biquad IIR filter CRN correctly processes 5 input cycles and produces outputs of 200, 235, 1264, 1237, and 1852 (in scaled units), matching the hand-calculated expected values exactly across all 5 trials.

The use of a large fast-to-slow rate ratio (10000:1 for Fibonacci, 10000:0.0001 for the biquad) ensures that all data operations finish before the next control step fires, which is the key design principle that makes this stochastic reaction networks compute reliably.